

Università di Salerno

DIPARTIMENTO DI INGEGNERIA DELL'INFORMAZIONE E MATEMATICA
APPLICATA



Algoritmi e Protocolli per la Sicurezza Project Work - Gruppo 20

Nome e Cognome	Matricola	E-Mail Istituzionale
Omar Silano	IE22700120	o.silano@studenti.unisa.it
Antonio Vitale	IE22700259	a.vitale132@studenti.unisa.it

WP1 - Modello	4
1.1 Definizione e Motivazione del Sistema Elettorale	4
1.2 - Attori Onesti del Sistema	4
1.3 - Funzionalità	4
1.4 - Interazione delle Entità	5
1.5 - Possibili Attaccanti	6
1.6 - Proprietà	8
1.7 - Completeness	9
1.8 - Assunzioni di Fiducia	9
WP2 - Soluzione	11
2.1 - Fase 1: Setup	12
2.1.1 - Generazione delle chiavi dell'autorità elettorale	12
2.1.2 - Configurazione del quesito referendario	12
2.1.3 - Predisposizione della whitelist	12
2.2 - Fase 2: Autenticazione e Rilascio Token	13
2.2.1 - Richiesta di autenticazione	13
2.2.2 - Verifica e generazione token	13
2.2.3 - Caratteristiche strutturali del token	13
2.3 - Fase 3: Espressione e raccolta del voto	14
2.3.1 - Preparazione della scheda elettorale (C_i)	14
2.3.2 - Composizione e invio del messaggio (M_i)	14
2.3.3 - Validazione, Burning del token e Ricevuta (R_i)	15
2.4 - Fase 4: Scrutini e pubblicazione	16
2.4.1 - Chiusura delle urne e mescolamento	16
2.4.2 - Decifratura e conteggio	16
2.4.3 - Pubblicazione e Verificabilità Universale	16
2.5 - Distribuzione e Gestione delle chiavi	17
2.6 - Meccanismo di Verifica Individuale	17
2.7 - Scelte progettuali e Compromessi	19
WP3 - Analisi della sicurezza	20
3.1 - Autenticità	20
3.1.1 - Analisi	20
3.1.2 - Limiti	20
3.2 - Unicità	21
3.2.1 - Analisi	21
3.2.2 - Limiti residui	21
3.3 - Segretezza e Pseudoanonimato	21
3.3.1 - Analisi della confidenzialità del voto	21
3.3.2 - Analisi dello pseudoanonimato e del rischio di collusione	22
3.3.3 - Limiti residui	22
3.4 - Integrità	22
3.4.1 - Analisi dell'integrità del quesito	23

3.4.2 - Analisi dell'integrità delle schede in transito.....	23
3.4.3 - Analisi dell'integrità dell'urna.....	23
3.4.4 - Limiti residui.....	23
3.5 - Verificabilità Individuale.....	23
3.5.1 - Analisi.....	23
3.5.2 - Limiti residui.....	24
3.6 - Verificabilità Universale.....	24
3.6.1 - Analisi.....	24
3.6.2 - Limiti residui: il problema della correttezza dello scrutinio.....	25
3.7 - Resistenza alla Coercizione.....	25
3.8 - Analisi delle Vulnerabilità e Resistenza agli Attacchi.....	26
3.8.1 - Chosen-Plaintext Attack (CPA).....	26
3.8.2 - Chosen-Ciphertext Attack (CCA).....	26
3.8.3 - Replay Attack (Attacchi di Riproduzione).....	26
3.8.4 - Length Extension Attack (LEA).....	27
WP4 - Implementazione e Prestazioni.....	28
4.1 - Sviluppo.....	28
4.2 - Scelte implementative.....	28
4.3 - Analisi delle prestazioni.....	29
4.3.1 - Tempi di esecuzione.....	29
4.3.2 - Dimensione dei Messaggi.....	29
4.4 - Scalabilità.....	30
4.5 Resilienza agli Attacchi.....	30
4.5.1 Chosen-Plaintext Attack (CPA).....	31
4.5.2 - Chosen-Ciphertext Attack (CCA) e Malleabilità.....	32
4.5.3 - Replay Attack e Double-Voting.....	32
4.5.4 - Length Extension Attack (LEA).....	33
4.5.5 Token Contraffatti (Violazione dell'Autenticazione).....	34

WP1 - Modello

In questo primo capitolo, ci concentreremo sulla definizione formale e concettuale del nostro sistema di voto elettronico. Esamineremo in primo luogo il sistema elettorale che si è scelto di progettare; in secondo luogo, analizzeremo gli attori onesti che compongono l'infrastruttura, delineando i loro scopi legittimi. Infine, ci concentreremo sulle minacce che possono interferire sul sistema elettorale: definiremo quindi il modello di minaccia (*Threat Model*), andando ad identificare i potenziali avversari per analizzare le loro risorse e i loro obiettivi, e le proprietà di sicurezza fondamentali che il sistema deve preservare per garantire la correttezza della consultazione elettorale.

1.1 Definizione e Motivazione del Sistema Elettorale

Il sistema è progettato per gestire una consultazione referendaria di tipo binario (Sì/No).

La scelta del referendum binario è dettata da precisi vincoli di efficienza computazionale ed efficacia crittografica. Riducendo lo spazio del voto a due sole opzioni, la dimensione delle schede cifrate e la complessità degli algoritmi di decifratura e conteggio vengono ridotte al minimo. Questo ci permette di ottimizzare le prestazioni complessive del protocollo e la latenza dei messaggi fin dalla fase di modellazione.

1.2 - Attori Onesti del Sistema

All'interno del nostro scenario elettorale operano diverse entità legittime. Ciascun attore è caratterizzato da responsabilità specifiche e agisce nel rispetto delle regole stabilite:

- **Amministratore del Referendum:** L'ente che ha il compito di inserire il quesito binario del referendum, definire la lista dei requisiti per avere diritto al voto e stabilire i tempi di apertura e chiusura delle urne;
- **Elettori:** Gli utenti aventi diritto al voto che desiderano esprimere la propria preferenza;
- **Sistema di Autenticazione:** Un'entità preposta a verificare l'identità reale degli elettori attraverso strumenti esterni;
- **Autorità Elettorale (Server di Raccolta):** L'infrastruttura di raccolta voti che procede a raccogliere e produrre il risultato finale.

Nota: Gli attori "Sistema di Autenticazione" e "Autorità elettorale" devono essere separati per mantenere la segretezza. I due Attori devono comunicare tra loro per garantire che ogni elettore possa votare al più una volta.

1.3 - Funzionalità

Le funzionalità dell'applicazione si dividono in tre blocchi principali:

→ Setup

- ◆ *Creazione del Referendum:* Il sistema consente all'Amministratore del Referendum di configurare la sessione elettorale mediante l'inserimento del quesito referendario in formato testo, i requisiti per il diritto al voto e i parametri temporali.

- ◆ *Blindatura del Quesito*: il quesito viene protetto crittograficamente da modifiche o alterazioni, in modo da garantire che tutti gli elettori visualizzano e votino per la medesima domanda.
 - ◆ *Predisposizione della lista dei votanti*: Il sistema riceve e configura una lista di requisiti che rendono l'elettore avente diritto al voto (es. età >18).
- **Accesso alla piattaforma**
- ◆ *Autenticazione Certificata*: Il sistema fornisce un'interfaccia di accesso che porta ad una piattaforma (simulata) di verifica d'identità forte. Il superamento di questa fase garantisce l'accesso alla piattaforma di voto anonimo (o pseudo anonimo), scollegando formalmente l'identità anagrafica dalla scheda di voto.
- **Voto e Scrutinio**
- ◆ *Voto*: L'elettore può selezionare una delle due opzioni disponibili. Il sistema convalida il voto, assicurandosi che l'utente non abbia già votato. Una volta registrato il voto è definitivo, garantendo la massima stabilità e integrità del database delle urne.
 - ◆ *Scrutinio Pubblico e Verificabile*: Alla chiusura della finestra temporale il sistema interrompe la ricezione delle schede, esegue il conteggio e pubblica il risultato finale.

1.4 - Interazione delle Entità

Per garantire l'equilibrio tra l'autenticità degli elettori, l'unicità del voto e la segretezza della preferenza, le entità che compongono il sistema comunicano seguendo un protocollo a fasi rigorosamente sequenziali, basato sul principio della separazione dei ruoli (*Separation of Duties*). Le interazioni si sviluppano in quattro fasi principali:

1. **Inizializzazione (Setup)**: L'Amministratore del Referendum interagisce parallelamente con due infrastrutture. Da un lato, trasmette al Sistema di Autenticazione i requisiti per generare l'elenco degli aventi diritto (whitelist). Dall'altro, comunica con l'Autorità Elettorale (Server di Raccolta) per caricare il quesito referendario, impostare la finestra temporale e predisporre i parametri di sicurezza necessari a proteggere la riservatezza delle schede elettorali.
2. **Autenticazione e Rilascio Autorizzazione**: L'Elettore si connette al Sistema di Autenticazione e fornisce le proprie credenziali reali. Il sistema verifica l'identità dell'utente e la sua presenza all'interno della whitelist. Se il controllo ha esito positivo, il Sistema di Autenticazione rilascia all'Elettore una credenziale di autorizzazione anonima.
Nota bene: in questa fase non vi è alcuno scambio di preferenze elettorali; il Sistema di Autenticazione si limita a certificare il diritto a partecipare.
3. **Espressione e Raccolta del Voto**: L'Elettore, dal proprio client locale, protegge la propria preferenza in modo che sia incomprensibile al server e la invia all'Autorità Elettorale, allegando la credenziale di autorizzazione ottenuta in precedenza. Il Server di Raccolta valida la credenziale per confermare che il mittente è autorizzato e non ha già votato. Una volta validata, la credenziale viene annullata (per prevenire il double-voting) e la scheda viene inserita nell'urna virtuale. *Cruciale per la Segretezza*: In questa fase, l'Autorità Elettorale e il Sistema di Autenticazione non comunicano in modo diretto e sincrono. Il Server di Raccolta si fida della validità della credenziale senza bisogno di conoscere a quale identità anagrafica corrisponda.
4. **Scrutinio (Tally) e Pubblicazione**: Terminato il periodo di voto, l'Autorità Elettorale chiude l'urna. Procede quindi all'elaborazione delle schede protette e pubblica il risultato aggregato.

A questo punto, sia l'Amministratore del Referendum che gli Elettori possono interagire in sola lettura con i dati resi pubblici per verificare l'integrità del conteggio.

1.5 - Possibili Attaccanti

In questa sezione vengono identificati e discusse le tipologie di avversari interessati a compromettere la sicurezza e la regolarità del referendum binario, specificando per ciascuno gli obiettivi e le risorse/capacità a disposizione:

- Gestore disonesto del Server di Raccolta (Autorità Elettorale):
 - *Obiettivo*: Tenta di alterare il conteggio finale delle preferenze ("Sì" o "No") modificando i dati memorizzati, inserire schede fraudolente sfruttando l'astensionismo, oppure violare la segretezza cercando di associare i voti ricevuti ai mittenti.
 - *Risorse/Capacità*: Elevate. Ha il controllo logico e fisico sul database delle urne digitali, l'accesso diretto ai log di rete del server di raccolta e la possibilità di manipolare l'ordine di ricezione o l'immagazzinamento delle schede.
 - *Implicazioni*: Invalidazione del risultato elettorale, perdita totale di fiducia nel sistema democratico e grave compromissione della segretezza del voto (violazione della privacy degli elettori).
 - *Mitigazioni*: Il sistema adotta tecniche crittografiche che impediscono al server di leggere i voti in chiaro e consentono il conteggio sui dati protetti. La proprietà di Verificabilità Universale garantisce che qualsiasi alterazione sia rilevabile in fase di scrutinio.
- Gestore disonesto del Sistema di Autenticazione (Identity Provider):
 - *Obiettivo*: Tenta di compromettere lo pseudoanonimato degli elettori colludendo con il server di raccolta (ad esempio scambiando o incrociando i log temporali degli accessi con quelli di ricezione delle schede). In alternativa, tenta di generare token di autorizzazione falsi per consentire il voto a soggetti non presenti nella lista degli aventi diritto.
 - *Risorse/Capacità*: Elevate. Possiede l'accesso completo al database delle identità anagrafiche reali (CIE) e controlla i meccanismi di emissione dei certificati di autenticazione.
 - *Implicazioni*: Inquinamento dell'urna con voti illegittimi (alterando l'esito del referendum) o, in caso di collusione, completa rottura della segretezza del voto che espone gli elettori a ritorsioni o profilazione.
 - *Mitigazioni*: Il sistema impone una rigorosa separazione delle responsabilità (*Separation of Duties*). L'Identity Provider certifica il diritto al voto rilasciando un token (o un'attestazione crittografica), ma non ha mai accesso alla scheda compilata. Contro la collusione, il collegamento temporale tra autenticazione e voto viene spezzato, e l'emissione di token falsi è prevenuta dalla tracciabilità pubblica degli accessi (senza rivelare l'identità) e dalla corrispondenza esatta con la *whitelist* iniziale.
- Elettore malintenzionato o coercibile:

- *Obiettivo*: Tenta di violare l'unicità del voto cercando di esprimere più di una preferenza (double-voting), di ripudiare il proprio voto a consultazione conclusa, oppure di produrre una prova digitale della propria scelta ("Sì"/"No") da mostrare a soggetti esterni (coercizione o compravendita del voto).
- *Risorse/Capacità*: Limitate. Non ha accesso alle infrastrutture server. Dispone esclusivamente delle proprie credenziali di autenticazione, del controllo del proprio dispositivo client e della facoltà di deviare intenzionalmente dalle istruzioni standard del software locale per esporre dati crittografici parziali.
- *Implicazioni*: Alterazione del peso elettorale del singolo individuo e instaurazione di un mercato del voto o di dinamiche di coercizione, che minano alla base la libertà della consultazione.
- *Mitigazioni*: Il double-voting è impedito dal Server di Raccolta, che invalida le autorizzazioni già spese. La coercizione è mitigata fornendo una "ricevuta" progettata matematicamente per dimostrare l'inclusione della scheda nell'urna, ma strutturalmente incapace di rivelare la scelta in chiaro a terzi.
- Avversario sul canale di comunicazione (Attaccante di rete):
 - *Obiettivo*: Tenta di intercettare il traffico di rete tra i client e i server per carpire informazioni utili alla de-anonimizzazione o per manipolare/bloccare i messaggi in transito (attacchi alla disponibilità del sistema durante la finestra temporale di voto).
 - *Risorse/Capacità*: Medie. Ha la capacità di monitorare i flussi di dati e iniettare o ritardare pacchetti, ma non possiede le chiavi necessarie per violare la cifratura.
 - *Implicazioni*: Interruzione del servizio elettorale (impedendo agli aventi diritto di votare), compromissione della riservatezza dei dati in transito o alterazione silente dei pacchetti diretti al server.
 - *Mitigazioni*: Adozione di protocolli di comunicazione sicuri unita a meccanismi di cifratura applicati direttamente sul dispositivo locale dell'elettore. In questo modo, l'attaccante ottiene solo dati illeggibili. Gli attacchi alla disponibilità del servizio vengono contrastati da un'infrastruttura di rete adeguatamente dimensionata.
- Elettore non idoneo al diritto di voto:
 - *Obiettivo*: Tenta di votare il Referendum nonostante il suo profilo non risulti idoneo ai requisiti di voto scelti per l'elezione.
 - *Risorse/Capacità*: *Limitate. Non possiede credenziali valide registrate all'interno della whitelist d'istituto. Opera esclusivamente modificando i parametri locali del proprio applicativo client nel tentativo di ingannare il Sistema di Autenticazione o forzare l'accettazione di una richiesta non autorizzata.*
 - *Implicazioni*: Partecipazione di soggetti non autorizzati, con conseguente inquinamento dei risultati e violazione del principio di legittimità della consultazione.
 - *Mitigazioni*: L'accesso alla piattaforma di espressione del voto è strettamente vincolato al superamento dell'autenticazione. Il Sistema di Autenticazione nega il rilascio del token crittografico se i requisiti anagrafici non combaciano con la whitelist. Il Server di Raccolta rifiuta automaticamente qualsiasi scheda sprovvista di un token di autorizzazione matematicamente valido.

1.6 - Proprietà

Si definiscono le proprietà che si vogliono preservare in presenza di attacchi al sistema elettorale:

- **Autenticità:** Solo gli elettori legittimi, provvisti di credenziali valide e crittograficamente verificabili, possono partecipare al voto. Sebbene nel nostro scenario l'infrastruttura di verifica dell'identità sia simulata (rappresentando un'astrazione di un *Identity Provider* reale), il protocollo richiede formalmente che il sistema si interfacci con tale modulo di Autenticazione. Questo passaggio è necessario per convalidare l'appartenenza dell'utente alla lista degli aventi diritto (*whitelist*) e per emettere il token di autorizzazione indispensabile per abilitare l'accesso alla fase di espressione della preferenza.
- **Unicità:** Ciascun elettore può esprimere al più un singolo voto valido. Il protocollo deve impedire tassativamente qualsiasi tentativo di sottomissione multipla (*double-voting*). In virtù delle scelte progettuali adottate per questo sistema, l'esclusione di una funzionalità di modifica o revoca comporta che il voto, una volta convalidato e registrato nel Server di Raccolta, sia definitivo e immutabile, blindando la proprietà di unicità fin dall'atto dell'invio.
- **Segretezza:** La preferenza espressa dal singolo elettore ("Sì" o "No") deve rimanere strettamente confidenziale. Questa proprietà garantisce che, analizzando il traffico di rete o i dati immagazzinati nell'urna, sia impossibile associare un voto alla reale identità anagrafica dell'elettore. Tale garanzia si fonda sul principio della separazione dei ruoli e rimane inviolabile a condizione che venga rispettata l'assunzione di non-collusione tra il Sistema di Autenticazione e l'Autorità Elettorale (dettagliata nella Sezione 1.8).
- **Integrità:** I voti regolarmente espressi e registrati all'interno dell'urna digitale non devono poter essere alterati, soppressi, rimossi o sostituiti. Allo stesso modo, l'integrità deve estendersi al quesito referendario configurato nella fase di setup, impedendo modifiche non autorizzate al testo della domanda. L'immutabilità dei dati memorizzati garantisce che il risultato finale rifletta esattamente la somma delle reali volontà espresse dal corpo elettorale.
- **Verificabilità Individuale:** Ogni elettore deve poter controllare autonomamente, attraverso strumenti matematici o ricevute crittografiche, che il proprio voto sia stato correttamente ricevuto dal Server di Raccolta e incluso nel conteggio finale delle schede. Tale processo di verifica deve essere strutturato in modo da non generare alcuna prova esportabile della preferenza specifica espressa ("Sì"/"No"), preservando la sicurezza semantica (CCA-security) della scheda: un osservatore esterno non può distinguere se il ciphertext contenga un Sì o un No, né può alterarlo in modo predicibile.
- **Verificabilità Universale:** Il risultato finale della consultazione deve essere pubblicamente verificabile da chiunque, inclusi osservatori esterni, enti regolatori o l'opinione pubblica. Chiunque deve avere la possibilità di esaminare i dati aggregati e le relative prove crittografiche per validare matematicamente la correttezza dello scrutinio e del conteggio finale, escludendo la necessità di riporre una fiducia incondizionata nell'onestà dei gestori del sistema.

Tra le proprietà non garantite vediamo:

- **Revocabilità:** Il sistema non prevede alcuna funzionalità che consenta all'elettore di annullare, modificare o sostituire la propria preferenza in una fase successiva all'invio della scheda all'Autorità Elettorale, prima della chiusura delle urne. Nonostante questo aiuterebbe

in caso di errore dell'utente, la sua implementazione porterebbe rischi di instabilità al database, e richiede un legame tra identità e scheda di voto che romperebbe lo pseudo-anonimato.

- **Resistenza alla Coercizione Perfetta (Fisica e Locale):** Pur garantendo l'assenza di ricevuta coercitiva per prevenire la compravendita del voto a posteriori, il sistema, operando in modalità remota, non può garantire la segretezza qualora un coercitore sia fisicamente presente alle spalle dell'elettore durante l'espressione del voto, o nel caso in cui il dispositivo client sia compromesso da malware in grado di registrare lo schermo o i tasti premuti.
- **Resistenza alla Collusione tra Entità:** Per mantenere il protocollo efficiente e scalabile, il sistema rinuncia a garantire la segretezza nel caso specifico in cui il Sistema di Autenticazione e l'Autorità Elettorale agiscano in modo disonesto accordandosi per incrociare i rispettivi log (come meglio specificato nella Sezione 1.8 - Assunzioni di Fiducia).

1.7 - Completeness

La proprietà di *Completeness* definisce il comportamento nominale del sistema nell'ipotesi in cui tutte le parti coinvolte agiscano in conformità con le specifiche del protocollo, senza deviazioni malevole:

- Se un elettore legittimo, regolarmente incluso nella whitelist degli aventi diritto, esegue la procedura di accesso, il Sistema di Autenticazione rilascia un'attestazione valida.
- Se l'elettore esprime una preferenza ("Sì" o "No") utilizzando un'attestazione valida, il Server di Raccolta convalida la scheda, la inserisce nell'urna virtuale e restituisce una ricevuta crittografica di avvenuta inclusione.
- Alla chiusura della finestra temporale, il Server di Raccolta elabora i dati memorizzati in modo esatto, pubblicando un risultato finale che corrisponde matematicamente alla somma reale delle schede elettroniche registrate.

1.8 - Assunzioni di Fiducia

Per garantire un ragionevole compromesso tra efficienza computazionale, trasparenza e segretezza, il modello si fonda su precise assunzioni di fiducia verso alcune componenti del sistema. Tali assunzioni comportano rischi residui esplicitamente accettati:

- **Non-collusione tra le entità di gestione (Separation of Duties):**
 - *Assunzione:* Si assume che il Sistema di Autenticazione e l'Autorità Elettorale (Server di Raccolta) operino in domini amministrativi strettamente separati e non colludono tra loro.
 - *Motivazione:* Questa assunzione è necessaria per mantenere i costi computazionali e la latenza del protocollo a livelli sostenibili, evitando l'uso di primitive crittografiche eccessivamente pesanti per nascondere totalmente i metadati di rete.
 - *Rischio Residuo:* Qualora gli amministratori delle due entità si accordassero per incrociare i rispettivi log di sistema (es. orari di emissione dei token e orari di

ricezione delle schede), lo pseudo-anonimato degli elettori potrebbe essere compromesso, portando a una potenziale de-anonimizzazione dei voti.

- **Integrità del dispositivo locale dell'elettore (Client parzialmente fidato):**

- *Assunzione:* Si assume che il dispositivo utilizzato dall'elettore per esprimere la preferenza non sia compromesso da malware critici, spyware o keylogger.
- *Motivazione:* Il sistema elettorale non ha alcun controllo sull'ambiente di esecuzione locale dell'utente e non può garantire la totale affidabilità preventiva.
- *Rischio Residuo:* Un client infetto potrebbe ignorare la scelta reale dell'utente, cifrando e inviando a sua insaputa un voto differente prima dell'applicazione dei protocolli di protezione, oppure potrebbe registrare la preferenza in chiaro eludendo la segretezza.

Per chiarire le dinamiche di comunicazione appena descritte e l'isolamento tra i domini di competenza, la figura seguente illustra graficamente il principio di *Separation of Duties* su cui si fonda l'intera architettura.

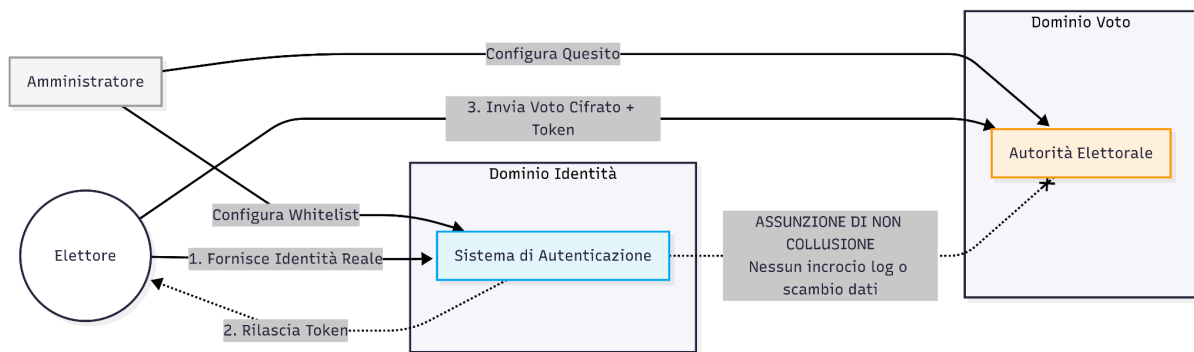


Figura 1: Diagramma di contesto e barriera di non-collusione tra il Sistema di Autenticazione (Dominio Identità) e l'Autorità Elettorale (Dominio Voto).

WP2 - Soluzione

In questo secondo capitolo, passeremo dalla definizione del modello alla sua concreta implementazione tecnica. Tradurremo le proprietà di sicurezza e le interazioni descritte nel WP1 in un protocollo crittografico operativo, strutturato in quattro fasi sequenziali: inizieremo con l'impostazione dell'infrastruttura (**Setup**), proseguiremo con il rilascio delle credenziali di accesso (**Autenticazione e Rilascio del Token**), passeremo all'espressione della preferenza da parte dell'elettore (**Espressione del Voto**) e concluderemo con il conteggio sicuro dei risultati (**Scrutinio e Pubblicazione**).

L'obiettivo è trasformare il modello teorico in un sistema robusto, capace di bilanciare le esigenze crittografiche con l'efficienza operativa richiesta dal protocollo.

Prima di analizzare in dettaglio le singole fasi operative e le primitive crittografiche adottate, la seguente figura fornisce una mappa visiva completa dell'intero ciclo di vita del protocollo, evidenziando le interazioni temporali tra tutte le entità coinvolte.

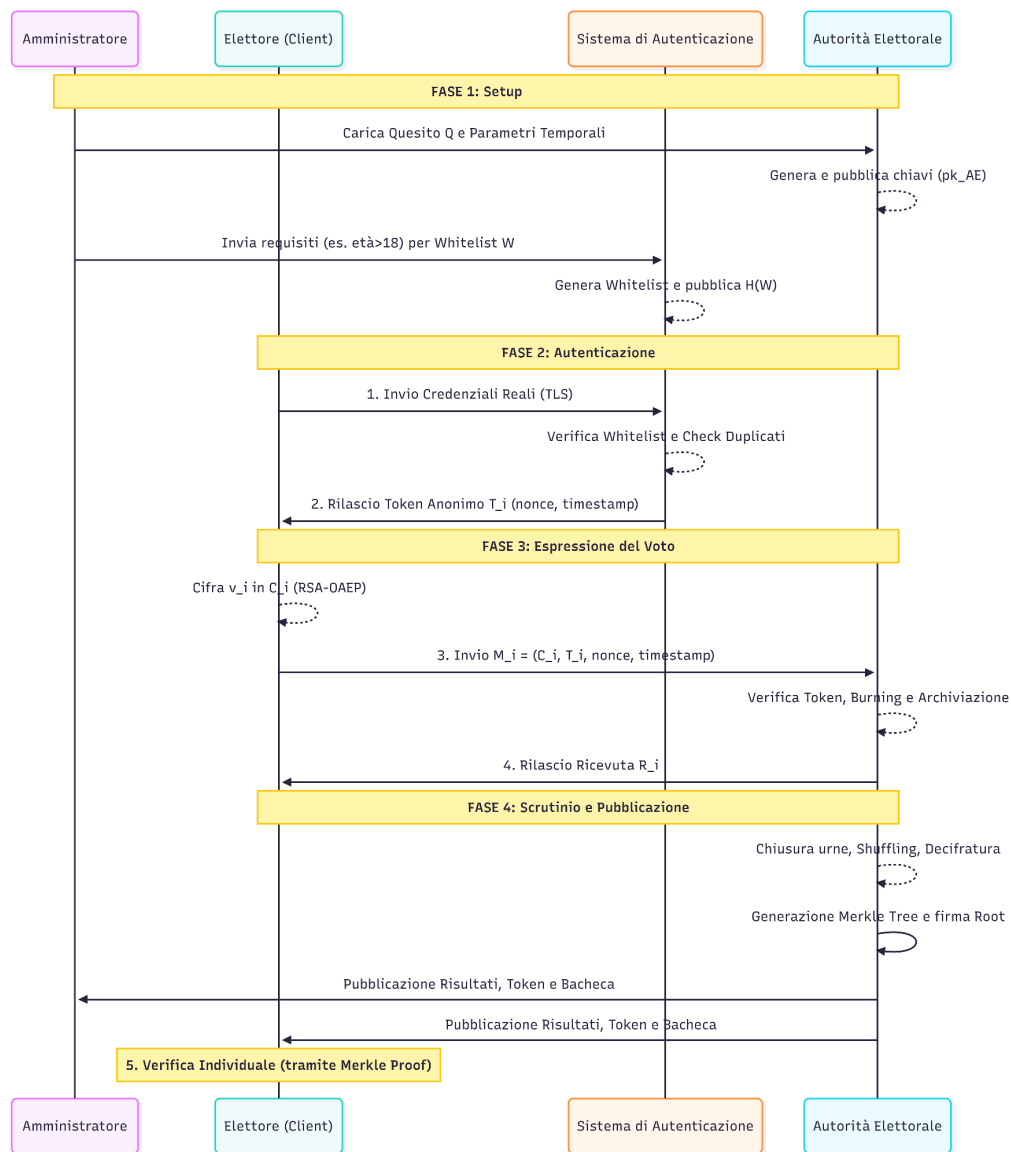


Figura 2: Diagramma di sequenza delle quattro fasi operative del sistema di voto elettronico.

2.1 - Fase 1: Setup

La fase di Setup costituisce le fondamenta di fiducia dell'intero sistema. In questo momento, l'Autorità Elettorale (AE) e l'Amministratore definiscono i parametri crittografici e le regole del Referendum binario, garantendo che ogni passaggio successivo sia verificabile e protetto da alterazioni.

2.1.1 - Generazione delle chiavi dell'autorità elettorale

L'Autorità Elettorale (AE) esegue l'algoritmo di generazione delle chiavi asimmetriche RSA. Questa operazione è fondamentale per garantire la confidenzialità delle schede durante l'intero svolgimento della consultazione: la chiave pubblica, infatti, permetterà agli elettori di cifrare le proprie preferenze, rendendole incomprensibili a chiunque durante il transito in rete; la chiave privata, invece, rimarrà segreta e sarà l'unico strumento capace di decifrare le schede, permettendo all'Autorità di procedere allo scrutinio solo a urne chiuse.

Il processo di generazione segue le seguenti fasi:

- Si scelgono due grandi numeri primi p e q tali che il modulo $n=p \cdot q$ abbia una lunghezza di almeno 2048 bit, garantendo così uno standard di sicurezza adeguato contro i tentativi di fattorizzazione;
- Si calcola la funzione di Eulero $\varphi(n) = (p-1)(q-1)$;
- Si sceglie l'esponente pubblico e (es. 65537) tale che $\gcd(e, \varphi(n)) = 1$;
- Si calcola la chiave privata d tale che $e \cdot d \equiv 1 \pmod{\varphi(n)}$;
- Pubblica la coppia $(pk_{AE} = (n, e))$ e mantiene segreta $(sk_{AE} = (n, d))$.

La chiave pubblica pk_{AE} viene pubblicata su una bacheca pubblica verificabile prima dell'apertura delle urne.

2.1.2 - Configurazione del quesito referendario

L'amministratore carica il testo del quesito Q sul sistema. Per garantire l'integrità del quesito, viene calcolato un hash crittografico H_Q tramite la funzione SHA256:

$$H_Q = \text{SHA}_{256}(Q \parallel \text{timestamp_apertura} \parallel \text{timestamp_chiusura})$$

H_Q viene firmato digitalmente dall'Amministratore con la propria chiave privata sk_{Admin} , utilizzando RSA con padding, e pubblicato. Qualsiasi alterazione successiva del testo Q è rilevabile confrontando l'hash calcolato con H_Q firmato.

2.1.3 - Predisposizione della whitelist

Tramite un canale di comunicazione sicuro e autenticato (es. un'interfaccia amministrativa protetta da TLS), il Sistema di Autenticazione riceve dall'Amministratore del Referendum i requisiti R (es. "età > 18", "nazionalità italiana"). Il Sistema di Autenticazione costruisce la Whitelist W degli elettori aventi diritto, verificando le credenziali di ciascuno rispetto a R . La whitelist non viene resa pubblica per garantire protezione della privacy; al suo posto, il Sistema di Autenticazione pubblica il numero totale degli elettori iscritti e l'impronta crittografica della lista $H(W)$. Questo parametro servirà a

verificare che il numero di voti finali non ecceda quello degli aventi diritto legittimi e a dimostrare che la lista non abbia subito alterazioni.

2.2 - Fase 2: Autenticazione e Rilascio Token

Questa fase ha lo scopo cruciale di risolvere il problema della separazione tra identità e voto, garantendo il principio di segretezza (pseudoanonimato). Il processo si basa sul rilascio di un "lasciapassare" crittografico univoco per ogni elettore.

2.2.1 - Richiesta di autenticazione

L' i -esimo elettore E_i si connette al Sistema di Autenticazione (SA) tramite un canale sicuro TLS (Transport Layer Security) e presenta le proprie credenziali reali. L'utilizzo del protocollo TLS previene che eventuali attaccanti di rete possano intercettare la sessione e rubare le credenziali in transito.

2.2.2 - Verifica e generazione token

Il Sistema di Autenticazione verifica che l'elettore sia presente nella whitelist ($E_i \in W$) e che non abbia già fatto richiesta di un token in precedenza. Superati i controlli, il sistema genera il token crittografico T_i :

$$T_i = \text{Sign_RSA-PSS}(sk_SA, \text{SHA-256}(\text{nonce}_i || \text{timestamp}))$$

dove:

- nonce_i è un valore di entropia imprevedibile ottenuto tramite CSPRNG (Cryptographically Secure Pseudorandom Number Generator).
- timestamp è il momento di emissione del token da parte del Sistema di Autenticazione .

Il Sistema Elettorale registra internamente la coppia (id_i, nonce_i) per impedire l'emissione di un secondo token allo stesso elettore, ma non trasmette id_i all'Autorità Elettorale in nessuna fase del protocollo. Al termine dell'operazione, all'elettore vengono consegnati i parametri $(\text{nonce}_i, \text{timestamp}, T_i)$.

2.2.3 - Caratteristiche strutturali del token

Il certificato crittografico così generato è progettato per garantire tre caratteristiche architetturali fondamentali, che costituiranno la base per l'analisi di sicurezza successiva:

- **Non falsificabilità:** Essendo firmato con sk_SA , la sua autenticità è verificabile da chiunque conosca la chiave pubblica pk_SA utilizzando la tripla $(\text{nonce}_i, \text{timestamp}, T_i)$.
- **Non riutilizzabilità:** il riutilizzo viene impedito dall'Autorità Elettorale tramite un registro dei token già spesi. Il nonce unico nonce_i garantisce che due token non siano mai identici, ma è il controllo lato Autorità Elettorale a rendere impossibile la presentazione multipla.

- **Anonimato rispetto all'Autorità Elettorale:** l'Autorità Elettorale riceve esclusivamente la tripla $(nonce_i, timestamp, T_i)$ e verifica la firma calcolando $SHA-256(nonce_i || timestamp)$ con pk_{SA} . Essendo l'identificativo id_i non incluso nel payload e mai trasmesso, l'Autorità Elettorale rimane strutturalmente cieca rispetto all'identità dell'elettore

2.3 - Fase 3: Espressione e raccolta del voto

Ottenuto il proprio "lasciapassare" crittografico anonimo, l'elettore procede con l'espressione della propria preferenza. L'obiettivo di questa fase è duplice: proteggere il contenuto del voto da chiunque non sia l'Autorità Elettorale (AE) e fornire all'elettore una ricevuta in grado di garantire la verificabilità individuale, pur prevenendo ogni forma di coercizione.

2.3.1 - Preparazione della scheda elettorale (C_i)

L'elettore E_i , dal proprio client locale, esprime la preferenza $v_i \in \{0, 1, 2\}$ (0 = No, 1 = Sì, 2 = Scheda nulla o non valida) in risposta al quesito Q. Prima che il dato lasci il dispositivo dell'elettore, il software client provvede a cifrare il voto v_i , utilizzando la chiave pubblica dell'Autorità Elettorale pk_{AE} .

Per prevenire attacchi di tipo semantico e garantire la non-malleabilità (CCA-Security), il sistema utilizza l'algoritmo di cifratura asimmetrica RSA accoppiato allo schema di padding OAEP (Optimal Asymmetric Encryption Padding). Il padding inserisce elementi di casualità nel processo, assicurando che cifrando la medesima preferenza (es. "Sì") due volte, si ottengano sempre ciphertext diversi.

La scheda cifrata C_i si calcola come:

$$C_i = Enc_{RSA-OAEP}(pk_{AE}, v_i)$$

2.3.2 - Composizione e invio del messaggio (M_i)

L'applicazione client compone il messaggio definitivo M_i unendo la scheda segreta C_i al certificato di autorizzazione (il token) ottenuto nella Fase 2. Il pacchetto risultante è così composto:

$$M_i = (C_i, nonce_i, timestamp, T_i)$$

Dal punto di vista strutturale, l'applicazione client si occupa di incapsulare in un unico payload dati di natura diversa, unendo il materiale crittografico appena generato con le credenziali preesistenti. La figura seguente illustra l'architettura logica del pacchetto risultante:

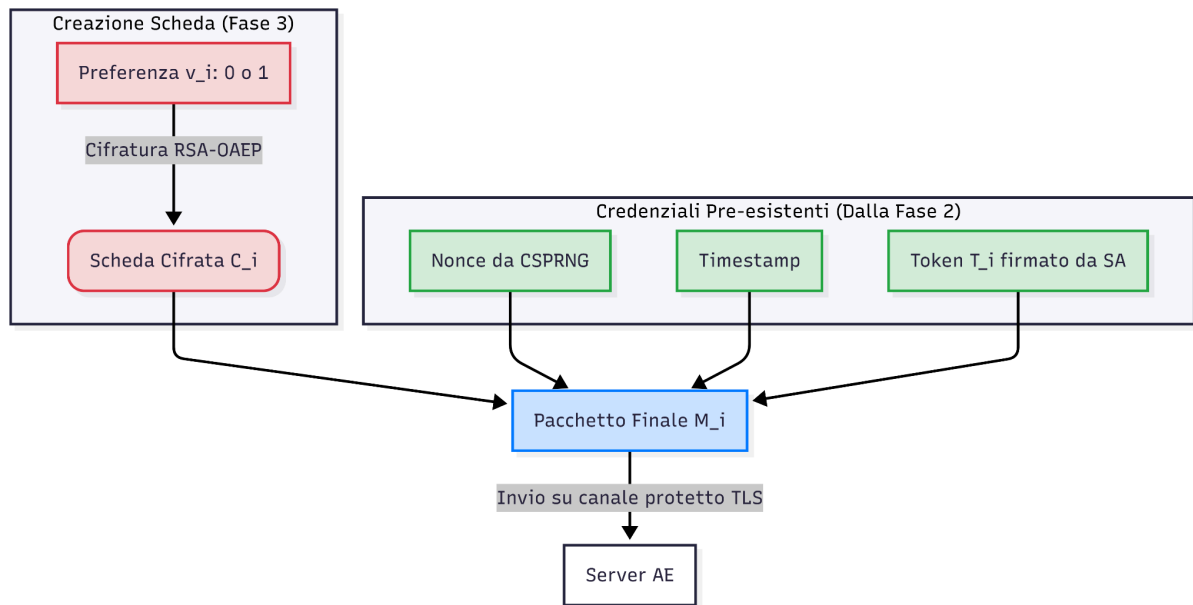


Figura 3: Anatomia logica del messaggio M_i . I dati rossi rappresentano le informazioni sensibili cifrate, i dati verdi rappresentano i parametri pubblici o verificabili.

L'elettore si collega al server dell'Autorità Elettorale tramite un nuovo canale sicuro TLS e trasmette il messaggio M_i . L'utilizzo del TLS impedisce la manipolazione in transito del payload crittografico, prevenendo l'inserimento di voti spazzatura da parte di eventuali attaccanti di rete.

2.3.3 - Validazione, Burning del token e Ricevuta (R_i)

L'Autorità Elettorale riceve il messaggio M_i dell'elettore tramite un canale TLS e, per preservare la segretezza garantita da RSA-OAEP, posticipa la verifica della validità formale della scelta (ovvero che sia 0 o 1) alla fase di scrutinio a urne chiuse. Qualora un elettore malintenzionato inviasse un crittogramma contenente "spazzatura", questo verrà annullato in fase di spoglio, esattamente come una scheda cartacea alterata.

Alla ricezione di M_i , l'Autorità Elettorale si limita ad eseguire i seguenti controlli logici propedeutici all'archiviazione:

1. **Verifica dell'Autenticità:** L'AE calcola l'hash della stringa $\{nonce_i || timestamp\}$ e utilizza la chiave pubblica del Sistema di Autenticazione (pk_{SA}) per validare la firma del token T_i .
2. **Verifica anti-Double-Voting:** L'AE controlla il proprio registro interno dei token spesi. Se l'impronta crittografica del token $H(T_i)$ è già presente, il messaggio viene rigettato per evitare il Double-Voting dell'elettore. Altrimenti, l'AE inserisce $H(T_i)$ nel registro, rendendo il "lasciapassare" associato non più utilizzabile in futuro (Burning).
3. **Deposito nell'urna:** Superati i controlli, la scheda cifrata C_i viene archiviata nell'urna elettronica U .

Ad operazione conclusa, l'AE genera e restituisce all'elettore una ricevuta crittografica R_i :

$$R_i = \text{Sign}(sk_{AE}, H(C_i || timestamp_{inserimento}))$$

La ricevuta R_i permette a E_i di verificare individualmente che il proprio voto è stato incluso nell'urna, rispondendo al requisito di Verificabilità Individuale definito nel WP1.
Al contempo, non rivela il contenuto di v_i a terzi: un eventuale coercitore otterrebbe solo la prova della partecipazione al Referendum binario, non la preferenza espressa.

2.4 - Fase 4: Scrutini e pubblicazione

Conclusa la finestra temporale di votazione, il sistema transita nella fase di scrutinio. L'obiettivo di questa fase è calcolare il risultato esatto del referendum e renderlo universalmente verificabile, preservando al contempo il totale anonimato delle preferenze espresse.

2.4.1 - Chiusura delle urne e mescolamento

Al raggiungimento di *timestamp_chiusura*, stabilito durante la Fase 1 dall'Amministratore del Referendum, l'AE smette di accettare nuovi messaggi M_i .

Prima di procedere all'apertura delle schede, l'AE esegue un'operazione di mescolamento. Questo passaggio è fondamentale per distruggere qualsiasi correlazione temporale tra l'ordine di arrivo dei messaggi sul server e l'ordine di decifratura delle schede, aggiungendo un ulteriore livello di protezione all'anonimato dell'elettore.

Fatto ciò, l'urna U viene finalmente congelata.

2.4.2 - Decifratura e conteggio

Successivamente al mescolamento, l'AE utilizza la propria chiave privata sk_{AE} , mantenuta segreta fino a quel momento, per decifrare l'intero set di schede C_i .

Ogni singola scheda in chiaro v_i viene ottenuta in risultato della seguente funzione:

$$v_i = Dec(sk_{AE}, C_i)$$

Durante questa operazione, l'AE effettua la validazione formale precedentemente posticipata:

- I testi in chiaro che corrispondono a $v_i \in \{0, 1, 2\}$ vengono registrati e categorizzati ai fini del conteggio ufficiale (0 = No, 1 = Sì, 2 = Scheda Bianca/Astensione).
- I testi in chiaro malformati (es. stringhe casuali o valori non ammessi al di fuori del set definito), derivanti da manomissioni del payload crittografico, vengono scartati in via definitiva, in stretta analogia con le schede fisiche illeggibili o annullate in un seggio tradizionale.

2.4.3 - Pubblicazione e Verificabilità Universale

Terminato lo spoglio, l'Autorità Elettorale pubblica ufficialmente i risultati sulla bacheca di sola lettura. Per soddisfare il requisito di Verificabilità Universale definito nel WP1 e garantire la massima trasparenza, l'AE non si limita a dichiarare il risultato, ma pubblica i seguenti elementi:

- Il numero totale di schede valide (Sì e No) e il numero di schede nulle.

- L'elenco pubblico di tutti i token $H(T_i)$ spesi, permettendo a chiunque di verificare che il numero totale dei votanti non ecceda quello degli aventi diritto e che nessuno abbia votato due volte.
- L'elenco finale dei voti in chiaro validi, nel loro ordine mescolato.
- La Merkle Root dell'urna: L'AE costruisce un *Merkle Tree* utilizzando tutti i crittogrammi C_i registrati come "foglie". L'hash radice (Merkle Root) viene calcolato, firmato digitalmente con la chiave segreta dell'Autorità Elettorale sk_{AE} e pubblicato.

La Merkle Root congela matematicamente lo stato dell'urna. Qualsiasi osservatore esterno può ricalcolare l'albero partendo dall'insieme U pubblicato; se la radice calcolata coincide con la Root firmata dall'AE, si ha la garanzia assoluta che nessuna scheda è stata aggiunta, alterata o soppressa post-chiusura.

In questo modo, ogni osservatore esterno dispone di tutti gli elementi crittografici e statistici per attestare la correttezza della consultazione referendaria, pur non potendo in alcun modo risalire all'identità di chi ha espresso la singola preferenza.

Nello specifico, qualsiasi osservatore esterno può matematicamente verificare:

- Che il numero di schede $|U|$ sia $\leq |W|$ (coerenza con la whitelist pubblicata in Setup).
- Che la firma dell'AE sul risultato sia valida tramite la chiave pubblica pk_{AE} .
- Che nessun token sia stato speso due volte, accertando l'assenza di duplicati nell'insieme dei token $H(T_i)$ pubblicati.

2.5 - Distribuzione e Gestione delle chiavi

Tabella 1: Riepilogo delle coppie di chiavi crittografiche asimmetriche e del loro ambito di utilizzo all'interno del protocollo.

Chiave	Generata da	Pubblica	Privata	Scopo
pk_{AE} / sk_{AE}	Autorità Elettorale	pk_{AE}	sk_{AE}	Cifratura schede / Decifratura scrutinio
pk_{SA} / sk_{SA}	Sistema di Autenticazione	pk_{SA}	sk_{SA}	Firma e verifica token
pk_{ADMIN} / sk_{ADMIN}	Amministratore	pk_{ADMIN}	sk_{ADMIN}	Firma quesito referendario

Tutte le chiavi pubbliche vengono pubblicate prima dell'apertura delle urne, consentendo a chiunque di verificare le firme in modo indipendente.

2.6 - Meccanismo di Verifica Individuale

Dopo la pubblicazione del risultato, l'elettore E_i può verificare che il proprio voto sia stato regolarmente conteggiato eseguendo i seguenti passaggi:

1. Recupera dalla bacheca pubblica il proprio crittogramma C_i ed il relativo *timestamp_inserimento*.
2. Ricalcola l'impronta $H(C_i \parallel \text{timestamp_inserimento})$ e utilizza la chiave pubblica dell'AE pk_{AE} per verificare matematicamente la validità della firma apposta sulla ricevuta R_i posseduta.
3. Estrapola dalla bacheca pubblica la Merkle Proof (il percorso crittografico di validazione) relativa al proprio C_i e ricalcola gli hash fino a ottenere la radice dell'albero.
4. Verifica che la radice ricalcolata combaci esattamente con la Merkle Root firmata e pubblicata dall'AE.

Per comprendere l'efficienza logaritmica di questa operazione, la figura seguente schematizza la struttura del Merkle Tree pubblicato sulla bacheca, evidenziando il set minimo di dati richiesto a un singolo utente per condurre la verifica:

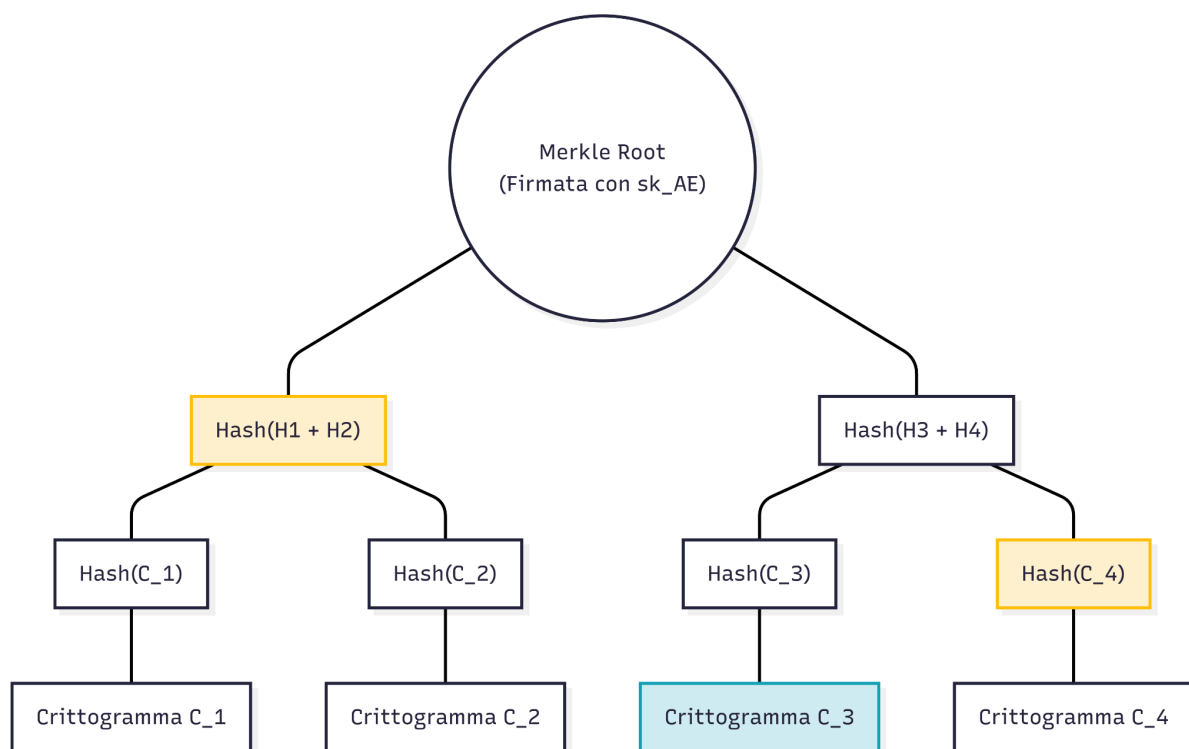


Figura 4: Esempio di verifica tramite Merkle Proof. L'elettore possessore del crittogramma C_3 (in azzurro) scarica unicamente gli hash "fratelli" (in giallo) necessari a ricalcolare la Merkle Root firmata dall'Autorità Elettorale.

Se i controlli hanno esito positivo, E_i ha la certezza matematica che il proprio voto cifrato è stato incluso nel conteggio. L'uso del Merkle Tree rende questa verifica estremamente scalabile: l'elettore non deve scaricare l'intera urna, ma solo una prova di dimensione logaritmica $O(\log N)$, rendendo l'operazione efficiente anche per Referendum con un elevato numero di voti registrati. Al contempo, poiché C_i è cifrato e randomizzato, la verifica non produce alcuna prova della preferenza espressa che sia esportabile a terzi, prevenendo così i tentativi di coercizione e preservando la non-malleabilità (CCA-security) garantita dall'uso del padding RSA-OAEP.

2.7 - Scelte progettuali e Compromessi

- **Cifratura Asimmetrica (RSA-OAEP) vs. Simmetrica / OTP:** La crittografia simmetrica soffre del problema della scalabilità nella distribuzione delle chiavi: richiederebbe all'AE di concordare in anticipo una chiave segreta con ciascuno degli n elettori tramite canali sicuri out-of-band, con una gestione $O(n)$ delle chiavi. Il One-Time Pad (OTP) offre *perfect secrecy* in senso information-theoretic, ma è impraticabile in questo contesto poiché eredita lo stesso problema di distribuzione e impone l'uso di chiavi monouso lunghe quanto il messaggio stesso. L'uso di RSA-OAEP risolve elegantemente il problema: l'AE pubblica una singola chiave pubblica pk_{AE} utilizzabile da tutti gli elettori in modo indipendente, garantendo *sicurezza computazionale* e resistenza semantica ad attacchi CCA.
- **Rinuncia alla Revocabilità del voto:** Come anticipato in WP1, garantire la revocabilità richiederebbe la persistenza di un collegamento tracciabile tra l'identità dell'elettore e la specifica scheda registrata nell'urna. Questo romperebbe il principio di pseudoanonimato. Il costo in termini di potenziale violazione della segretezza è stato giudicato superiore al beneficio offerto dalla revocabilità.
- **Assunzioni di Fiducia (Threat Model):**
 - **Segretezza inviolabile di sk_{AE} :** Il protocollo assume che la chiave segreta dell'AE sk_{AE} rimanga inviolata per l'intero ciclo di vita del sistema, garantendo la confidenzialità delle schede a lungo termine. Il suo utilizzo per decifrare l'urna avviene in un ambiente sicuro solo a votazioni concluse; una compromissione vanificherebbe irrimediabilmente la segretezza delle preferenze in transito.
 - **Non-collusione tra Entità (SA e AE):** Questa è la principale assunzione di fiducia e il limite strutturale più rilevante del sistema. Il Sistema di Autenticazione (SA) conosce il legame tra l'identificativo id_i e il *nonce* i generato; l'Autorità Elettorale (AE) conosce il legame tra il *nonce* i e la scheda cifrata C_i . Se SA e AE dovessero colludere incrociando i rispettivi log, il disaccoppiamento identità-voto verrebbe neutralizzato, infrangendo irrimediabilmente lo pseudoanonimato. Questo rischio dovrà essere valutato dettagliatamente nel WP3.
 - **Struttura Dati (Merkle Tree vs Blockchain):** Per garantire l'immutabilità dell'urna a fine votazione e una verificabilità individuale efficiente, si è scelto di implementare un Merkle Tree. Si è volutamente scartata l'ipotesi di adottare un'architettura Blockchain: il requisito di decentralizzazione del consenso tramite nodi peer-to-peer è superfluo nel nostro modello, che prevede già un'Autorità Elettorale istituzionale. L'uso del solo Merkle Tree permette di ottenere i medesimi benefici di inalterabilità del dato e scalabilità logaritmica, evitando gli enormi overhead computazionali e di latenza tipici delle reti DLT.

WP3 - Analisi della sicurezza

In questo terzo capitolo analizziamo la sicurezza del protocollo progettato in WP2 rispetto al modello di minaccia definito in WP1. Per ciascuna proprietà di sicurezza identificata — autenticità, unicità, segretezza, integrità, verificabilità individuale e verificabilità universale — valutiamo se il protocollo la soddisfa nelle condizioni dichiarate, evidenziando i limiti strutturali e discutendo i rischi residui. Infine, sottoporremo l'architettura a una verifica di robustezza contro i principali vettori di attacco crittografico e di rete (CPA, CCA, Replay e LEA).

3.1 - Autenticità

La proprietà di autenticità richiede che solo gli elettori legittimi, ovvero quelli inclusi nella whitelist W costruita dal Sistema di Autenticazione (SA), possano ottenere un token valido e dunque partecipare al Referendum.

3.1.1 - Analisi

Il Sistema di Autenticazione verifica l'identità dell'elettore tramite canale TLS e controlla la sua presenza in W prima di emettere il token T_i . Il token è una firma digitale RSA-PSS su

$SHA-256(\text{nonce}_i || \text{timestamp})$, prodotta con la chiave segreta sk_{SA} .

L'Autorità Elettorale (AE) accetta esclusivamente schede accompagnate da un token la cui firma sia verificabile tramite la chiave pubblica pk_{SA} : un attaccante privo di sk_{SA} non è in grado di forgiare un token valido, pena la violazione della resistenza alla falsificazione esistenziale delle firme RSA.

Un elettore non idoneo che tenta di modificare i parametri del proprio client non può superare questo controllo: la firma è calcolata su una stringa che include un nonce_i generato lato SA e mai reso noto all'esterno nella sua associazione con l'identità reale. La proprietà di autenticità è quindi soddisfatta sotto l'assunzione standard che il parametro di sicurezza della chiave RSA-2048 sia adeguato (ovvero che la fattorizzazione del modulo n_{SA} sia computazionalmente infattibile per l'avversario).

3.1.2 - Limiti

Un gestore disonesto del SA potrebbe emettere token a soggetti non presenti in W , aggirando il controllo. Questo attacco è però parzialmente mitigato dalla pubblicazione della lista $H(W)$ e del numero degli aventi diritto al voto $|W|$: qualsiasi discrepanza tra il numero di token emessi e $|W|$ sarebbe rilevabile durante la fase di scrutinio universale.

Tuttavia, questo controllo previene solo l'emissione di token in eccesso rispetto alla popolazione totale. Se il SA disonesto sfruttasse l'astensionismo fisiologico, potrebbe emettere token illegittimi mantenendo il totale $\leq |W|$ ("sostituzione degli astenuti").

Poiché il SA non pubblica le identità di chi ha votato per tutelare la privacy, questo attacco silenzioso rappresenta un rischio residuo ineliminabile, accettato nell'assunzione di fiducia dichiarata nel WP1.

3.2 - Unicità

La proprietà di unicità richiede che ciascun elettore possa esprimere al più un voto valido, impedendo qualsiasi tentativo di sottomissione multipla (double-voting).

3.2.1 - Analisi

Il meccanismo di prevenzione del double-voting opera su due livelli complementari:

- **Lato SA:** il Sistema di Autenticazione registra la coppia $(id_i, nonce_i)$ e nega l'emissione di un secondo token allo stesso elettore. Un elettore legittimo può quindi ottenere al più un token per sessione referendaria.
- **Lato AE:** al ricevimento del messaggio $M_i = (C_i, nonce_i, timestamp, T_i)$, l'Autorità Elettorale controlla il registro dei token spesi tramite l'impronta $H(T_i)$. Se il token è già presente, il messaggio viene rigettato (burning). Il nonce univoco $nonce_i$ garantisce che token distinti producano impronte $H(T_i)$ distinte, rendendo infattibile la collisione intenzionale.

La combinazione dei due livelli rende strutturalmente impossibile il double-voting anche qualora un elettore riuscisse a intercettare il proprio token prima del burning: il secondo tentativo di invio fallirebbe comunque al controllo lato AE.

3.2.2 - Limiti residui

Il meccanismo ipotizza che il registro dei token spesi nell'AE sia persistente e atomico: un'implementazione che non garantisca atomicità nelle operazioni di lettura/scrittura potrebbe, in linea teorica, permettere la doppia presentazione in caso di richieste concorrenti ravvicinate. Questo è un rischio implementativo da gestire in WP4, non un limite crittografico del protocollo.

3.3 - Segretezza e Pseudoanonimato

La proprietà di segretezza richiede che la preferenza del singolo elettore non sia in alcun modo associabile alla sua identità reale. Come ampiamente discusso nel Modello (WP1), il nostro sistema garantisce questa proprietà in modo forte contro attaccanti esterni, ma presenta un limite architetturale volontario noto come "Assunzione di Non-Collusione": la segretezza viene meno nel caso peggiore in cui SA e AE colludano incrociando i propri log. Questa è la proprietà più critica e il rischio residuo più rilevante dell'intera architettura.

3.3.1 - Analisi della confidenzialità del voto

Le schede sono cifrate con RSA-OAEP sotto la chiave pubblica pk_{AE} . La sicurezza CCA di RSA-OAEP garantisce che, senza chiave segreta sk_{AE} , nessun avversario è in grado di distinguere il ciphertext di un voto 'Sì' da quello di un voto 'No' con probabilità non trascurabile. In particolare:

- La randomizzazione del padding OAEP assicura che due cifrature dello stesso voto producano ciphertext distinti, impedendo attacchi basati sulla confrontabilità dei crittogrammi.
- La proprietà di non-malleabilità impedisce a un avversario attivo di trasformare un ciphertext valido in uno diverso con un risultato noto, prevenendo manipolazioni delle schede in transito.

La chiave segreta sk_{AE} resta segreta fino alla chiusura delle urne: sul piano crittografico, tuttavia, l'entità che detiene fisicamente sk_{AE} in memoria ha matematicamente la capacità di decifrare i crittogrammi in qualsiasi momento. Non esiste quindi un vincolo crittografico strutturale che impedisca all'AE una decifratura prematura, ma solo un vincolo procedurale. Questo rischio residuo viene mitigato dall'assunzione di fiducia (dichiarata nel WP1) verso l'Autorità Elettorale, la quale si impegna a utilizzare la chiave esclusivamente a urne chiuse nella fase formale di scrutinio.

3.3.2 - Analisi dello pseudoanonimato e del rischio di collusione

Il disaccoppiamento identità-voto è realizzato architetturalmente: il SA conosce $(id_i, nonce_i)$ ma non C_i ; l'AE conosce $(nonce_i, C_i)$ ma non id_i . In assenza di collusione, nessuna singola entità dispone delle informazioni necessarie per collegare un'identità a una preferenza.

In caso di collusione tra SA e AE, la de-anonimizzazione non richiede un'analisi euristica o probabilistica basata sui timestamp: il SA possiede il legame $(id_i, nonce_i)$ e l'AE possiede il legame $(nonce_i, C_i)$. L'incrocio di questi due database utilizzando $nonce_i$ come chiave primaria condivisa è un'operazione deterministica che produce direttamente la tripla $(id_i, nonce_i, C_i)$, violando lo pseudoanonimato. Questo è il limite strutturale più grave del sistema, esplicitamente dichiarato come rischio residuo accettato in WP1 e WP2.

3.3.3 - Limiti residui

Il protocollo introduce una mitigazione parziale della correlazione temporale attraverso il mescolamento delle schede prima della decifratura (sezione 2.4.1 di WP2). Il mescolamento distrugge la correlazione tra l'ordine di ricezione dei messaggi e l'ordine di apertura delle schede, rendendo più difficile l'attacco per correlazione temporale. Tuttavia, questa mitigazione non è crittograficamente forte: la correlazione rimane possibile se i log di ricezione delle singole connessioni TLS vengono conservati con timestamp precisi.

3.4 - Integrità

La proprietà di integrità richiede che i voti registrati nell'urna non possano essere alterati, soppressi o sostituiti dopo la loro registrazione, e che il quesito referendario non possa essere modificato dopo la fase di setup.

3.4.1 - Analisi dell'integrità del quesito

Il quesito Q è protetto da un hash crittografico $H_Q = \text{SHA-256}(Q \parallel \text{timestamp_apertura} \parallel \text{timestamp_chiusura})$, firmato con la chiave segreta sk_Admin e pubblicato. Qualsiasi modifica al testo di Q produrrebbe un hash diverso, rilevabile da chiunque confronti il valore pubblicato con quello calcolato localmente. La sicurezza si riduce alla resistenza alla seconda preimmagine (Second Preimage Resistance) di SHA-256 — che impedisce a un attaccante di trovare un quesito modificato Q' che produca lo stesso hash del quesito originale Q già pubblicato — e alla non-falsificabilità della firma RSA dell'amministratore, entrambe solidamente fondate.

3.4.2 - Analisi dell'integrità delle schede in transito

Le schede viaggiano su canale TLS, che garantisce autenticazione del server, riservatezza e integrità in transito tramite MAC. Un attaccante di rete non può modificare il contenuto di M_i senza che la modifica venga rilevata e la connessione interrotta. Questo protegge dall'attacco di un network adversary attivo.

3.4.3 - Analisi dell'integrità dell'urna

Una volta depositata, la scheda cifrata C_i è tutelata dalla ricevuta $R_i = \text{Sign}(sk_AE, H(C_i \parallel \text{timestamp_inserimento}))$. Se l'AE alterasse o eliminasse una scheda dall'urna, l'elettore titolare di R_i potrebbe rilevarlo nella fase di verifica individuale, non trovando il proprio C_i nell'insieme U pubblicato. Alterazioni silenziose dell'urna sono quindi rilevabili dagli elettori stessi.

3.4.4 - Limiti residui

A differenza dei sistemi di voto elettronico tradizionali basati su bacheche piatte, il nostro protocollo implementa un meccanismo strutturale per prevenire l'alterazione o la soppressione selettiva delle schede post-chiusura. Poiché l'AE è tenuta a strutturare l'insieme U come un Merkle Tree e a firmare digitalmente la Merkle Root al termine delle votazioni, l'urna assume le caratteristiche di un registro Append-Only e Tamper-Evident. Se l'AE tentasse di scartare o alterare i voti degli elettori in modo silenzioso, la struttura logaritmica degli hash propagherebbe l'errore fino a modificare inesorabilmente la Root finale. Questo rende la manomissione matematicamente evidente e rilevabile a posteriori non solo dalla singola vittima, ma da chiunque ricalcoli l'albero. Il limite residuo si riduce quindi alla sola omissione di inserimento durante la fase di voto (se il server scarta attivamente il pacchetto M_i non fornendo la ricevuta R_i), che richiede però un disservizio evidente lato client.

3.5 - Verificabilità Individuale

La proprietà di verificabilità individuale richiede che ogni elettore possa accertare autonomamente che il proprio voto sia stato incluso nel conteggio, senza che tale verifica riveli la preferenza espressa.

3.5.1 - Analisi

Il meccanismo di verifica individuale descritto in WP2 (sezione 2.6) si articola nei seguenti controlli:

1. L'elettore recupera dalla bacheca pubblica il proprio crittogramma C_i e il relativo *timestamp*.
2. Ricalcola l'impronta e utilizza la chiave pubblica dell'AE pk_{AE} per verificare matematicamente la validità della firma apposta sulla ricevuta R_i .
3. Estrapola dalla bacheca pubblica la Merkle Proof (il percorso crittografico di validazione) relativa al proprio C_i e ricalcola gli hash fino a ottenere la radice dell'albero.
4. Verifica che la radice ricalcolata combaci esattamente con la Merkle Root firmata e pubblicata dall'AE.

La verifica è crittograficamente solida: la ricevuta R_i e la Merkle Root sono firme RSA, il cui controllo richiede solo pk_{AE} e i valori pubblici. L'uso della Merkle Proof rende la procedura altamente efficiente ed elegantemente scalabile, richiedendo lo scaricamento di un set logaritmico di dati e non dell'intera urna. La non-rivelabilità della preferenza è preservata perché C_i è un ciphertext RSA-OAEP: la sola osservazione del crittogramma, così come la sua verifica crittografica, non fornisce informazioni sul bit v_i a un osservatore esterno o a un potenziale coercitore.

3.5.2 - Limiti residui

La verifica individuale è condizionata alla correttezza della decifratura da parte dell'AE: se l'AE decifra le schede correttamente ma manipola il conteggio aggregato dei voti in chiaro (ad esempio, sommando erroneamente le preferenze valide), l'elettore non può rilevarlo attraverso la sola verifica del proprio C_i . Questa debolezza riguarda tuttavia più la verificabilità universale che quella individuale, ed è discussa nella sezione successiva.

3.6 - Verificabilità Universale

La proprietà di verificabilità universale richiede che chiunque — osservatori esterni, enti regolatori, cittadini — possa verificare la correttezza del risultato finale senza dover riporre fiducia incondizionata nei gestori del sistema.

3.6.1 - Analisi

L'AE pubblica sulla bacheca pubblica i seguenti elementi (sezione 2.4.3 di WP2):

- Il numero totale di schede valide e nulle.
- L'elenco pubblico di tutti i token $H(T_i)$ spesi, per la verifica dell'unicità e della coerenza con $|W|$.
- L'elenco finale dei voti in chiaro validi, nel loro ordine mescolato.
- La Merkle Root dell'urna, calcolata a partire dai crittogrammi C_i e firmata digitalmente con la chiave segreta dell'Autorità Elettorale sk_{AE} .
- La firma dell'AE sul risultato aggregato.

Grazie a questi dati, un osservatore esterno può verificare:

1. Che $|U| \leq |W|$: il numero di schede non supera quello degli aventi diritto.
2. Che la firma dell'AE sul risultato sia valida sotto pk_{AE} , attestando l'autenticità della dichiarazione.
3. Che i token spesi siano distinti, garantendo l'assenza di double-voting a livello aggregato.
4. Che il conteggio aggregato dei voti validi (Sì e No) sia coerente con il numero totale di schede presenti in U e con i voti in chiaro pubblicati nell'ordine mescolato, consentendo a chiunque di ricalcolare il risultato finale e confrontarlo con quello dichiarato dall'AE.
5. Che la Merkle Root pubblicata sia matematicamente coerente: qualsiasi osservatore può ricalcolare l'intero albero partendo dall'insieme U dei crittogrammi esposti in bacheca. Se la radice calcolata coincide con la Root firmata dall'AE, si ha la garanzia assoluta (radicata nella resistenza alle collisioni di SHA-256) che nessuna scheda sia stata aggiunta, alterata o soppressa post-chiusura.

3.6.2 - Limiti residui: il problema della correttezza dello scrutinio

Il punto strutturalmente più critico riguarda la correttezza della fase di decifratura e conteggio. Il sistema pubblica i voti in chiaro nel loro ordine mescolato, ma non fornisce una prova crittografica che ciascun voto in chiaro corrisponda alla corretta decifratura di un ciphertext presente in U .

In altre parole:

- L'AE potrebbe, in linea teorica, decifrare le schede correttamente e poi pubblicare una lista di voti in chiaro manipolata, mantenendo la firma valida (poiché firma il risultato finale, non i singoli voti).
- La corrispondenza tra l'insieme $\{C_i\}$ e l'insieme $\{v_i\}$ pubblicato è verificabile solo implicitamente e non può essere dimostrata a terzi senza compromettere l'intero impianto di sicurezza: la verifica richiederebbe il possesso di sk_{AE} , la quale tuttavia deve rimanere rigorosamente segreta (o essere distrutta mediante procedure certificate) anche dopo la chiusura dello scrutinio. Se sk_{AE} fosse resa pubblica, chiunque potrebbe decifrare i crittogrammi C_i esposti sulla bacheca pubblica e, incrociandoli con le ricevute R_i in possesso degli elettori, risalire al voto in chiaro di ciascuno, compromettendo a ritroso sia l'anonimato sia la resistenza alla coercizione dell'intera elezione.

3.7 - Resistenza alla Coercizione

Il sistema previene strutturalmente la vendita del voto o la coercizione a posteriori: l'elettore possiede infatti una ricevuta R_i che certifica esclusivamente la sua partecipazione al referendum, ma non rivela in alcun modo la preferenza espressa. Essendo R_i una firma su un crittogramma RSA-OAEP, un potenziale coercitore che entrasse in possesso della ricevuta e del crittogramma non potrebbe in alcun modo distinguere se il voto contenuto sia un 'Sì' o un 'No'.

Il limite residuo (comune a tutti i sistemi di voto da remoto) è la coercizione "alle spalle dell'elettore": un utente potrebbe collaborare attivamente con il coercitore mostrando il voto in chiaro sul proprio monitor prima della fase di cifratura, o condividendo lo schermo del proprio dispositivo. Questo attacco fisico si verifica in un perimetro esterno al protocollo crittografico (il client locale dell'elettore) e non può essere mitigato via software.

3.8 - Analisi delle Vulnerabilità e Resistenza agli Attacchi

Di seguito vengono analizzate le capacità di difesa del sistema elettorale rispetto a diverse tipologie di attacco:

3.8.1 - Chosen-Plaintext Attack (CPA)

In questo scenario, l'attaccante ha la facoltà di cifrare testi in chiaro a sua scelta. Nel contesto del nostro Referendum, lo spazio dei messaggi è estremamente limitato (le uniche opzioni valide sono "Sì" o "No"). Un attaccante potrebbe semplicemente cifrare queste due opzioni con la chiave pubblica dell'Autorità Elettorale (AE) e confrontarle con i ciphertext intercettati sulla rete per risalire alle preferenze espresse, violando irrimediabilmente la segretezza.

Mitigazione: L'adozione del padding RSA-OAEP mitiga completamente questa vulnerabilità. L'inserimento di padding randomizzato rende la cifratura intrinsecamente probabilistica: cifrare lo stesso voto "Sì" cento volte produrrà cento ciphertext C_i completamente diversi, garantendo la proprietà di sicurezza semantica.

3.8.2 - Chosen-Ciphertext Attack (CCA)

Un attaccante attivo intercetta, inietta o altera ciphertext scelti e osserva l'output del processo di decifratura (o semplicemente spera di generare un ciphertext valido per un testo alterato).

Mitigazione: Un attaccante malevolo potrebbe catturare il ciphertext C_i di un elettore e provare a manipolare i bit del pacchetto per alterarne il contenuto in chiaro (ad esempio, invertendo il voto espresso), sfruttando la proprietà di malleabilità tipica di alcune cifrature base. La specifica implementazione di RSA-OAEP protegge il sistema fornendo la proprietà di non-malleabilità: agendo come una procedura di verifica di integrità integrata al livello di cifratura, qualsiasi modifica non autorizzata ai bit del crittogramma corromperà la struttura attesa al momento della rimozione del padding. Questo causa il fallimento della decifratura e il conseguente rigetto della scheda da parte dell'Autorità Elettorale, senza fornire all'attaccante messaggi d'errore sfruttabili.

3.8.3 - Replay Attack (Attacchi di Riproduzione)

Un avversario passivo intercetta un messaggio autenticato e legittimo sulla rete pubblica, per poi reinviarlo tal quale in un momento successivo per produrre un effetto non autorizzato sul server (senza bisogno di decifrare o alterare nulla).

Mitigazione: Nel nostro scenario elettorale, un avversario potrebbe catturare l'intero pacchetto del voto di un utente (contenente il crittogramma e il relativo token di autorizzazione) e reinviarlo ripetutamente nel tentativo di far contabilizzare la medesima preferenza più volte. Questa problematica viene respinta attraverso i controlli applicati dall'Autorità Elettorale (AE). Al ricevimento del messaggio, l'AE verifica l'impronta crittografica del token $H(T_i)$ rispetto al proprio registro interno dei token spesi: l'accettazione della prima sottomissione invalida l'autorizzazione associata. Di conseguenza, i successivi tentativi di *replay* dello stesso pacchetto o token vengono deterministicamente identificati e rigettati.

3.8.4 - Length Extension Attack (LEA)

Questo è un attacco strutturale che affligge le funzioni di hash basate sulla costruzione iterativa di Merkle-Damgård (tra cui SHA-256, usata nel progetto). L'attaccante, conoscendo solo la lunghezza del messaggio originario e il suo hash, può continuare il processo di compressione aggiungendo un suffisso arbitrario e calcolando un hash valido per il messaggio esteso, senza conoscere il contenuto segreto originario. Questo rende vulnerabili le implementazioni ingenui dei Message Authentication Code del tipo $MAC = H(k || m)$.

Mitigazione: Il sistema è strutturalmente immune a questa classe di attacchi per due ragioni fondamentali. In primo luogo, l'adozione del Merkle Tree per il congelamento dell'urna interrompe la natura sequenziale dei flussi di hashing tipici della vulnerabilità LEA, poiché i blocchi vengono accoppiati gerarchicamente ad albero. In secondo luogo, tutti i parametri sensibili (come il quesito referendario o la Merkle Root finale) vengono firmati digitalmente con chiavi asimmetriche (sk_{Admin} e sk_{AE}). Di conseguenza, pur ammettendo che un attaccante possa estendere arbitrariamente un payload e calcolarne l'hash esteso, non disporrebbe delle chiavi private necessarie per ricalcolare la firma digitale, rendendo la manipolazione immediatamente rilevabile in fase di verifica pubblica.

WP4 - Implementazione e Prestazioni

In questo quarto e ultimo capitolo viene presentata l'implementazione pratica del protocollo di voto elettronico progettato nel WP2 e discusso nel WP3. L'obiettivo è dimostrare la fattibilità tecnica della soluzione attraverso un ambiente simulato, analizzando al contempo le prestazioni computazionali, l'efficienza dei messaggi scambiati e la scalabilità del sistema.

4.1 - Sviluppo

Il protocollo è stato implementato interamente in linguaggio Python, simulando le interazioni tra i vari attori (Amministratore, Sistema di Autenticazione, Autorità Elettorale ed Elettori) tramite un'applicazione stand-alone in esecuzione locale. L'architettura del software è suddivisa in tre moduli principali:

- **voting_protocol.py**: Il cuore crittografico del sistema. Contiene l'implementazione formale delle quattro fasi del protocollo (Setup, Autenticazione, Voto, Scrutinio) e le funzioni di verifica (individuale e universale). Include inoltre l'implementazione personalizzata della struttura dati Merkle Tree.
- **simulation.py**: Uno script di benchmark automatizzato. Si occupa di orchestrare l'intera consultazione elettorale misurando accuratamente i tempi di esecuzione, le dimensioni dei payload e testando attivamente la resilienza del sistema contro attacchi specifici.
- **voting_cli.py**: Un'interfaccia interattiva a riga di comando (CLI) che permette all'utente di simulare passo-passo i ruoli dei diversi attori, verificando empiricamente le garanzie di sicurezza e il corretto avanzamento dello stato del sistema.

4.2 - Scelte implementative

Per garantire il rispetto delle proprietà di sicurezza descritte nei capitoli precedenti, il software si appoggia alla libreria standard cryptography, aderendo alle migliori pratiche del settore:

- **Chiavi asimmetriche**: Sono state utilizzate chiavi RSA a 2048 bit con esponente pubblico standard (65537) per tutte le entità.
- **Cifratura delle schede**: La protezione della segretezza è implementata tramite RSA-OAEP (Optimal Asymmetric Encryption Padding) utilizzando SHA-256 come funzione di hash. Questo garantisce la non-malleabilità (CCA-security) dei crittogrammi.
- **Firme digitali e Token**: L'autenticità dei token, del quesito referendario, delle ricevute e della Merkle Root è garantita dallo standard di firma RSA-PSS (Probabilistic Signature Scheme) basato su SHA-256, che offre solide garanzie contro attacchi di falsificazione.
- **Payload a lunghezza fissa**: Per ottimizzare l'efficienza e prevenire vulnerabilità di parsing, i pacchetti scambiati non utilizzano separatori, ma concatenazioni di byte a lunghezza fissa (es. il nonce è sempre 32 byte, il timestamp 8 byte e la firma RSA 256 byte).
- **Gestione delle Schede Bianche e Nulle**: Per garantire un'esperienza utente (UX) aderente alla realtà elettorale, il sistema gestisce in modo sicuro le intenzioni di voto non valide (es. inserimento di caratteri errati o invii a vuoto). Invece di generare un errore applicativo o rigettare l'elettore, il protocollo assegna a questi input un valore di fallback convenzionale (il

valore 2). Questo payload viene regolarmente cifrato con RSA-OAEP, imbustato nell'urna e conteggiato in fase di scrutinio come "Voto Nullo / Scheda Bianca". Questa scelta ingegneristica garantisce il diritto all'astensione attiva dell'elettore senza compromettere la stabilità del software.

4.3 - Analisi delle prestazioni

La valutazione delle prestazioni è stata condotta eseguendo la simulazione automatizzata **simulation.py** su una base di 10 elettori.

4.3.1 - Tempi di esecuzione

Come mostrato nella Tabella 2, i tempi di elaborazione dimostrano che l'uso della crittografia asimmetrica non introduce latenze bloccanti per l'esperienza utente o per le operazioni lato server.

Tabella 2: Tempi medi di esecuzione delle primitive crittografiche suddivisi per fase operativa (benchmark effettuato su un campione di 10 elettori).

FASE OPERATIVA	OPERAZIONE	TEMPO MEDIO
Fase 1: Setup	Generazione chiavi (Admin, SA, AE) e Whitelist	123.7 ms
Fase 2: Autenticazione	Generazione token (nonce + firma RSA-PSS)	0.495 ms
Fase 3: Voto (Client)	Cifratura della preferenza (RSA-OAEP)	0.052 ms
Fase 3: Voto (Server)	Verifica token, burning e rilascio ricevuta	0.644 ms
Fase 4: Scrutinio	Decifratura di 10 schede e calcolo Merkle Root	5.7 ms
Verifica Universale	Validazione firme, coerenza urne e ricalcolo root	0.109 ms
Verifica Individuale	Verifica firma ricevuta e Merkle Proof	0.063 ms

4.3.2 - Dimensione dei Messaggi

Oltre alle performance computazionali, l'ottimizzazione del traffico di rete è un requisito fondamentale. Come evidenziato nella Tabella 3, le dimensioni dei payload scambiati risultano

rigorosamente fisse ed estremamente contenute, rendendo il protocollo adatto anche a connessioni a bassa banda.

Tabella 3: Dimensionamento in byte dei payload e delle strutture dati crittografiche generate dal protocollo.

Componente	Contenuto	Dimensione
Scheda Cifrata (C_i)	Testo cifrato RSA-2048	256 byte
Token (T_i)	Nonce (32B) + Timestamp (8B) + Firma (256B)	296 byte
Messaggio Voto (M_i)	C_i (256B) + Token (296B)	552 byte
Merkle Root Firmata	Hash Root (32B) + Firma RSA (256B)	288 byte
Merkle Proof	Percorso di nodi per 10 elettori (4 nodi)	132 byte

4.4 - Scalabilità

Per verificare l'applicabilità del sistema su consultazioni più ampie, è stato eseguito un benchmark di scalabilità variando il numero di elettori (N) da 5 a 100. I risultati confermano le aspettative teoriche:

- **Costo costante $O(1)$ lato Client:** L'operazione di espressione del voto (cifratura e composizione del pacchetto) si mantiene stabile intorno a ~ 1.2 ms per singolo voto inviato indipendentemente dal numero totale di elettori.
- **Complessità Lineare $O(N)$ nello Scrutinio:** Essendo l'Autorità Elettorale tenuta a decifrare sequenzialmente ogni scheda, il tempo di spoglio cresce linearmente (da 1.75 ms per 5 elettori a 115.3 ms per 100 elettori). Tali valori rimangono comunque nell'ordine di decimi di secondo, dimostrando ampia fattibilità.
- **Complessità Logaritmica $O(\log N)$ nella Verifica:** Il tempo richiesto a un singolo elettore per validare la propria inclusione nell'urna (Merkle Proof) rimane quasi invariato (passando da 0.032 ms con $N=10$ a 0.0195 ms con $N=100$) grazie all'efficienza della struttura ad albero.

4.5 Resilienza agli Attacchi

Attraverso il modulo interattivo **voting_cli.py**, il sistema è stato sottoposto a routine di attacco automatizzate per verificare l'effettiva resilienza delle primitive crittografiche implementate (RSA-2048 e SHA-256).

L'interfaccia a riga di comando (CLI) sviluppata (Figura 5) permette di orchestrare la simulazione e lanciare in sequenza gli attacchi automatizzati. Di seguito si riporta la risposta del codice alle singole categorie di attacco previste dal Threat Model.

PROTOCOLLO DI VOTO ELETTRONICO	
1	Fase 1 – Setup referendum
2	Fase 2 – Rilascio token (singolo)
2b	Fase 2 – Rilascio token (tutti)
3	Fase 3 – Vota (singolo elettore)
3b	Fase 3 – Vota (tutti gli elettori)
4	Fase 4 – Scrutinio e pubblicazione
5	Verifica universale
6	Verifica individuale (Merkle Proof)
SIMULAZIONE ATTACCHI WP3	
7	Attacchi – CPA (Chosen-Plaintext)
8	Attacchi – CCA (Chosen-Ciphertext)
9	Attacchi – Replay Attack (Riproduzione)
10	Attacchi – LEA (Length Extension)
11	Attacchi – Double-Voting (Riuso token)
12	Attacchi – Token Contraffatto
13	Stato sessione
0	Esci

Figura 5: Menù principale del simulatore interattivo `voting_cli.py` utilizzato per il collaudo.

4.5.1 Chosen-Plaintext Attack (CPA)

La simulazione implementata nella funzione `do_attack_cpa(Session)` forza l'elettore a cifrare il medesimo voto in chiaro (1 per "Sì") due volte consecutive con la stessa chiave pubblica dell'Autorità Elettorale, avvalendosi del modulo di padding. L'output dimostra che i due ciphertext (C_1 e C_2) risultano strutturalmente discordanti in ogni byte, rendendo impossibile per un attaccante in ascolto dedurre il voto per confronto visivo o statistico.

```
=====
ATTACCO – Chosen-Plaintext Attack (CPA)
=====
Teoria: L'attaccante cifra tutti i voti possibili per creare un dizionario.
Se il sistema è deterministico i ciphertext corrisponderanno a quelli nell'urna.

Cifro il voto '1' (Sì) per due volte consecutive con la stessa chiave pubblica...
  C_1 = 35fb7fb63b3de9b755db64e10ae6869a...
  C_2 = 9982e8197768dafdf88d8192a45f3e7a...
[✓] I ciphertext sono completamente diversi!
[✓] Difesa riuscita: il padding probabilistico RSA-OAEP garantisce sicurezza CPA.
[✓] L'attaccante non può risalire ai voti confrontando i ciphertext.

Premi INVIO per continuare...
```

Figura 6: Output dell'attacco CPA. Generazione crittografica di due ciphertext completamente discordanti per il medesimo plaintext.

4.5.2 - Chosen-Ciphertext Attack (CCA) e Malleabilità

Nella funzione **do_attack_cca(Session)**, l'attaccante intercetta un ciphertext valido dall'urna e applica un'operazione XOR (^= 0xFF, equivale a 255 in decimale) sull'ultimo byte, rinviandolo all'Autorità Elettorale (AE) per lo scrutinio. Durante l'esecuzione di **rsa_decrypt**, il controllo di integrità interno dello standard OAEP rileva immediatamente la corruzione del blocco matematico prima ancora di restituire il plaintext. L'operazione solleva un'eccezione a livello di libreria crittografica, neutralizzando la malleabilità.

```
=====
ATTACCO - Chosen-Ciphertext Attack (CCA) e Malleabilità
=====
Teoria: L'attaccante intercetta il ciphertext C_i di 'elettore_000'.
Prova ad alterare un byte del ciphertext sperando di ribaltare il voto in chiaro.

C_originale (ultimi 16 B) = 2ec2a16c623329911a9b6e28a3d271b4
C_alterato  (ultimi 16 B) = 2ec2a16c623329911a9b6e28a3d2714b

L'attaccante invia il pacchetto malevolo; l'AE tenta la decifratura...
[✓] Errore catturato durante la decifratura: ValueError
[✓] Difesa riuscita: il controllo di integrità interno di RSA-OAEP
[✓] ha rilevato la corruzione strutturale, garantendo la non-malleabilità.

Premi INVIO per continuare... |
```

Figura 7: Neutralizzazione dell'attacco CCA. L'alterazione di un byte del crittogramma viene intercettata dai controlli di integrità del padding OAEP.

4.5.3 - Replay Attack e Double-Voting

Entrambi gli attacchi (**do_attack_replay** e **do_attack_double_voting**) ricompongono un messaggio M_i o accoppiano un nuovo ciphertext a un token T_i precedentemente processato. A runtime, la funzione **phase3_receive_vote** mitiga il Ciphertext Replaying invalidando l'accettazione di messaggi duplicati. Appena l'AE riceve la replica, le primitive di autenticazione del protocollo scattano ancor prima di spaccettare il voto, bloccando l'esecuzione con un'eccezione specifica.


```
=====
ATTACCO - Replay Attack (Riproduzione pacchetto)
=====
Teoria: L'avversario ha registrato il traffico di rete legittimo di 'elettore_000'.
Senza alterare nulla, ritrasmette il medesimo pacchetto M_i
per far valere il voto una seconda volta.

Tentativo di reinvio del pacchetto M_i all'Autorità Elettorale...
[✓] Bloccato correttamente: Token già speso: double-voting impedito.
[✓] Difesa riuscita: il set dei token spesi (token burning) funge da nonce univoco,
[✓] rigettando immediatamente pacchetti validi ma replicati.

Premi INVIO per continuare...|

=====
ATTACCO - Double-Voting
=====
Tentativo di riusare il token di 'elettore_000' (già speso)...
[✓] Bloccato correttamente: Token già speso: double-voting impedito.
```

Figure 8-9: Blocco dei tentativi di Replay Attack e Double-Voting tramite il meccanismo di invalidazione dei token di sessione (token burning).

4.5.4 - Length Extension Attack (LEA)

L'avversario concatena la stringa fittizia "||OPZIONE_FITTIZIA" al digest del quesito referendario originario (H_Q), calcola il nuovo hash tramite sha256() e tenta di farlo validare con la firma RSA preesistente dell'Admin. A runtime, la funzione rsa_verify fallisce e solleva l'eccezione InvalidSignature. Questo prova empiricamente che l'uso nativo dello schema RSA-PSS rende la firma strettamente vincolata all'esatta lunghezza e bit-sequence del digest originale, rigettando qualsiasi estensione.

```
=====
ATTACCO - Length Extension Attack (LEA)
=====
Teoria: nelle costruzioni deboli  $MAC = H(k || m)$  un attaccante può prolungare
l'hashing aggiungendo un suffisso senza conoscere k.

Simulazione: l'attaccante estende H_Q (quesito firmato dall'Admin)
e tenta di spacciare il nuovo hash usando la firma originale.

H_Q legittimo : 69020e2f60a326797f746c153d013c49...
H_Q alterato  : f45f12590a9669e25619703962d492ae...

L'attaccante usa la firma originale sul nuovo hash (non possiede sk_admin)...
[✓] La verifica crittografica ha rigettato il pacchetto (firma non valida).
[✓] Difesa riuscita: l'autenticazione è basata su RSA-PSS, non su MAC naïf.
[✓] Il LEA è strutturalmente inapplicabile al protocollo.

Premi INVIO per continuare...|
```

Figura 10: Fallimento del Length Extension Attack. La firma originale risulta crittograficamente non valida in seguito all'alterazione del digest.

4.5.5 Token Contraffatti (Violazione dell'Autenticazione)

L'attaccante genera un token simulando le lunghezze fisse del protocollo: 32 byte di nonce randomico (`os.urandom`), un timestamp e 256 byte di firma fittizia. All'apertura del payload nella Fase 3, l'Autorità Elettorale tenta di verificare la firma fittizia sui 256 byte finali rispetto al nonce tramite la chiave pubblica dell'Autorità di Setup (pk_{sa}). Poiché l'attaccante non possiede la chiave privata sk_{sa} , il controllo matematico fallisce istantaneamente, salvaguardando l'integrità dell'urna pubblica.

```
=====
ATTACCO - Token Contraffatto
=====
Genera un token con firma RSA casuale (non valida)...
  fake_nonce = d08616850ef14df8...
  fake_sig   = 21982970dff343e5...
[✓] Rigettato correttamente: Firma del token non valida.

Premi INVIO per continuare...
```

Figura 11: Blocco di un token contraffatto. Il sistema rileva l'invalidità della firma RSA generata casualmente dall'attaccante, rigettando immediatamente il pacchetto.